Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

BBR-With Adaptive Pacing Gain Rates (BBR-APG): An improved TCP-BBR Congestion Control Algorithm on RTT Fairness Design Problem

Charles Kihungi Njogu
Central South University
Wang Yang (yangwang@csu.edu.cn)
Central South University
Humphrey Waita Njogu
Kenya Institute for Public Policy Research and Analysis
Adrian Bosire
Kiriri Women's University of Science and Technology

Research Article

Keywords: BBR, Congestion Control, RTT fairness, Bottleneck, Bandwidth

Posted Date: July 3rd, 2023

DOI: https://doi.org/10.21203/rs.3.rs-3118820/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: No competing interests reported.

BBR-With Adaptive Pacing Gain Rates (BBR-APG): An improved TCP-BBR Congestion Control Algorithm on RTT Fairness Design Problem

Charles Kihungi Njogu[1†], Wang Yang[1*†], Humphrey Waita Njogu[2] and Adrian Bosire[3]

[1*]School of Computer Science and Engineering, Central South University, Changsha, 410083, People's Republic of China.
[2]Kenya Institute for Public Policy Research and Analysis, Nairobi, 56445-00200, Kenya.
[3]School of Computer Science and Technology, Kiriri Women's University of Science and Technology, Nairobi, 27235-00100, Kenya.

*Corresponding author(s). E-mail(s): yangwang@csu.edu.cn;
Contributing authors: charles.kihungi@zetech.ac.ke; hnjogu@kippra.or.ke; bosire.adrian@kwust.ac.ke;
†These authors contributed equally to this work.

Abstract

Google scientists published Bottleneck Bandwidth and Round-trip propagation time (BBR), a novel TCP congestion control algorithm variant. Unlike traditional congestion control algorithms, BBR adjusts its transmission rates by periodically measuring the network traffic based on round-trip propagation time (RTprop) and available bottleneck bandwidth (Btlbw). This results in BBR achieving higher delivery rates and minimizing the latency on our modern-day network infrastructure. However, some recent experimental evaluations and analyses have shown severe RTT (round-trip time) fairness concerns in the TCP-BBR algorithm. When two competing flows with different RTTs coexist on a shared bottleneck link, BBR tends to prefer and favor longer RTT flows,

therefore consuming more bandwidth share than short RTT counterparts, which can result in short RTT flows being starved even to death. This preference nature of the BBR in favor of long RTT flows results in an acute throughput imbalance and an extreme BBR and network vulnerability that malevolent individuals can easily exploit by increasing the delay (RTTs) in their flows to create unwarranted long RTTs on their TCP flows to unfairly obtain and enjoy the lion's share of the available bandwidth than the rest of users on the network. The BBR-With Adaptive Pacing Gain Rates (BBR-APG) algorithm is proposed in this study as an alternative solution to the TCP-BBR's RTT fairness problem. Our proposed approach adaptively adjusts the pacing gain rates of each TCP flow based on the bottleneck inflight queue status during the bandwidth estimation phase instead of the default fixed pacing gain rates as provided by the original version of the BBR algorithm, which only works perfectly when only a single flow is involved. With the BBR-APG algorithm, different RTT flows can fairly compete for the available bottleneck bandwidth, thus enhancing the RTT fairness issue that has been compromised in the original version of the BBR algorithm. Through our extensive simulation analysis on NS3, we identified that our proposed methodology improves RTT fairness by more than 18%, a reduction of more than 24% and 20% on packets retransmission rates and queuing delay, respectively, compared to other recently published BBR variants like an adaptive congestion window of BBR (BBR-ACW) on various network conditions.

Keywords: BBR, Congestion Control, RTT fairness, Bottleneck, Bandwidth

1 Introduction

Despite the research community effort towards diminishing packet loss problems during data transmission, our networks still experience massive packet loss, notably wireless networks [1, 2]. Packet losses may occur due to bottleneck buffer overflow (bufferbloat), network congestion, or network failures. In the mid-1980s, the Transmission Control Protocol (TCP) established network congestion control which considers packet loss as the only sign of a congested network, which is not always accurate in our modern networks [3]. TCP manages the congestion window (cwnd) from the server (sender), intending to control and govern the packets to be transmitted. Whenever no packet loss is noticed or detected, the cwnd surges exponentially until congestion is detected through duplicate acknowledgments (ACKs), and cwnd will be reduced by half to slow down data transmission [4, 5]. The majority of the current and existing TCP congestion control algorithms are loss-based, which read each packet loss as a sign of a congested network and cause bottleneck bufferbloat by keeping the bottleneck buffer always filled up [6, 7].

Google's research team published a novel TCP congestion control variant, Bottleneck Bandwidth and Round-trip propagation time (BBR), as a better alternative to classical loss-based congestion control algorithms. Contrary to

traditional TCP congestion control variants, BBR regularly measures Btlbw and RTprop of the network traffic and utilizes the measured results to adjust its transmission pattern. Therefore recording higher throughput with minimum transmission delay [8]. In addition, BBR uses bandwidth-delay product (BDP) estimates to reduce bufferbloat occurrences rather than applying a notation of packet loss as an indicator of a congested network. As a result, BBR has a significant comparative advantage over other TCP loss-based congestion control variants, especially in a lossy network setup. [9, 10]. BBR algorithm finds the optimal operating point (Kleinrock's optimal operating point) by measuring the highest possible delivery rate with a very minimal delay time [11], as demonstrated in Figure 1. Upon finding the optimal operating point, the BBR delivery rate remains constant while RTT starts increasing as a sign that the inflight packets are filling up the bottleneck buffer and the queue is starting to pile up. BBR has shown significant throughput and delay improvement compared to its predecessor algorithms [12, 13] such as Cubic [5] when implemented on various Google in-house servers and B4 network [9].

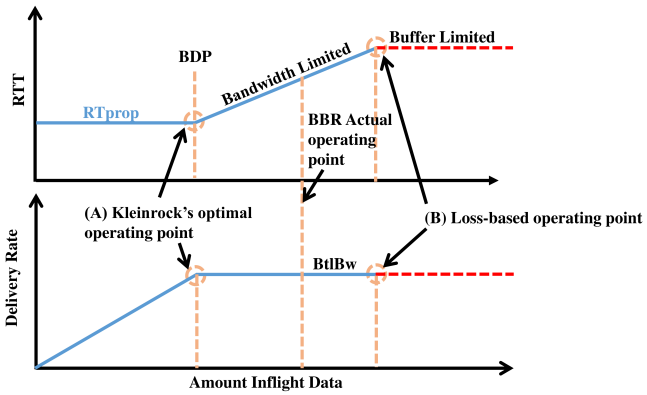


Fig. 1 A relationship between delivery rate, RTT, buffer inflight data and operating points based on [11]

BBR has four main phases to carry out its principal function: sending data at the fastest rate possible without creating a queue, as shown in Figure 2 [8, 14].

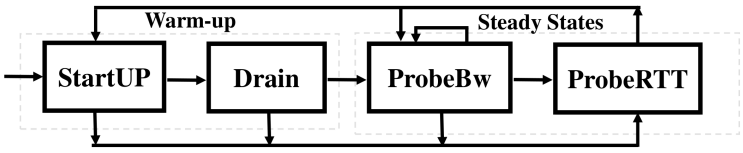


Fig. 2 BBR operational states.

- Startup

The data-sending rate is increased by BBR using a pacing gain value rate of 2/ln(2) until the actual bandwidth is identified. If the probing growth stops, BBR leaves the startup phase and moves into the drain phase, assuming the desired bandwidth has been reached.

- Drain

BBR applies an inverted startup pacing gain rate to reduce excessive data transmitted during the previous phase. When the inflight data becomes 1 BDP, BBR exits from this phase and enters probeBw.

- ProbeBw

The majority of BBR's time is spent on this phase. During this time, BBR is driven by its delivery capacity estimates. The estimated delivery capacity is then passed through the maximum bandwidth filter algorithm, which outputs the available bandwidth estimation. BBR applies a pacing rate of 1.25 times the measured bottleneck bandwidth to probe up for more bandwidth. This ends up generating the queue but getting more accurate capacity estimates and identifying great improvements in the capacity, this process only lasts for the RTT, and it also spends another RTT reducing the pacing rate to 0.75 times the bottleneck bandwidth to eliminate the bottleneck queue piling up on the buffer and on the link.

- ProbeRTT

If the new minimum RTT is not evaluated within 10 seconds during probeBw phase, BBR gets into the probeRTT phase. This phase measures the actual minimum RTT in the link; therefore, the maximum transmission is dropped to 4 packets to remove any queue in the bottleneck link. When a new minimum RTT value is obtained, it is updated as RTprop and maintained for the next 10 seconds. This phase only lasts for 0.2 seconds.

Encouraged by BBR's ever-promising performances, several recent studies have evaluated BBR on different network setups and application settings. Hock et al. [15], Ma et al. [16], and Zhang et al. [17] reported severe unfairness issues in TCP-BBR, specifically when two different TCP flows with varying RTTs coexist on a common bottleneck link.

We deployed the BBR algorithm on NS3 simulating environment for testing and analysis purposes based on [17–19]. After a series of scrutinization, we found that during the probebw phase, the BBR algorithm uses a fixed pacing gain rate of 1.25 to probe up for more available bandwidth and 0.75 pacing rate to probe down to eliminate both buffer and link queue generated during probe up session. These fixed default pacing rates applied by BBR work perfectly when only a single TCP flow is used. However, when different TCP flows with varying RTTs are involved during data transmission, BBR injects some extra data, which increases the delivery rate capacity during probe up

interval and causes the total transmission rate to exceed the available bottleneck bandwidth, resulting in excessive queue formation (bufferbloat). This is quite a surprising result because BBR was designed to eliminate bufferbloat occurrences; this issue was also reported by [20]. Our experimental analysis also revealed that the 0.75 probe down pacing rate adopted by BBR could not entirely drain the queue generated during probe up session; hence the persistent queue is formed over time on the bottleneck link. Since the estimated RTprop values of the long RTT flows are always bigger than that of short RTT ones; therefore long RTT flows will always occupy a larger ratio of the persistent queue. Each of the existing BBR flows loops this process repetitively; hence long RTT flows progressively occupy the more significant share of available bandwidth, forming an imbalanced and biased global synchronization, and short RTT flows are severely suppressed and even starved almost to death, creating an unfair distribution of this precious network resource. Fairness decreases as the RTT disparity between the competing flows widens.

From our analysis, we discovered that from time to time, BBR blindly probe up for more bandwidth without considering factors such as bottleneck buffer queue status or link congestion. When multiple flows with varying RTTs coexist on a shared bottleneck link, BBR tends to add extra bytes of data, which will cause an excessive queue, buffer overflow, and link congestion, which the default probe down pacing rate cannot drain or clear, and this goes down as the primary contributing factor to the unfairness of the various RTT flows in the BBR. This severe underlying BBR design choice, which works in favor of long RTT flows, creates a serious network flaw that some users can easily take advantage of by intentionally inflating the delay (RTTs) on their TCP flows, intending to create a skewed network throughput to acquire the lion's share of the available bandwidth than the rest of other network users.

Google is upgrading BBR; they have already released BBR version 2.0, which is in alpha edition [21] to address some of the major concerns observed by various experimental evaluations [9, 15, 22]. Several evaluations have already been conducted on the BBR Version 2.0, revealing that the RTT fairness problem was not addressed and still persists [23, 24].

The current study proposed a BBR-APG algorithm to enhance the fairness between TCP flows with varying RTTs in the BBR. Our proposed methodology adaptively obtains the probebw phase pacing rates based on the bottleneck inflight queue status. The inflight queue status size influences the probebw pacing rates instead of the default fixed pacing gain values of 1.25 and 0.75 as provided by the original version of the BBR algorithm, which only functions perfectly when a single TCP flow is used, which is unlikely in our current networks. BBR-APG was evaluated on the NS3 simulator. Our proposed algorithm's evaluation analysis performance results showed significant RTT fairness improvement from the standard BBR algorithm. Our Technique reported better performance results than other recently published BBR variants, ensuring various TCP flows with varying RTTs compete more equitably

for the available bottleneck bandwidth with minimal queuing delays, therefore increasing overall network throughput.

The remaining part of the paper is organized as follows. Section II introduces various related works on improving the RTT fairness problem in the TCP-BBR algorithm. A brief description of the motivation of our study is in Section III. Section IV presents our proposed algorithm. Section V describes NS3 experimental results and the analysis, and finally, Section 6 presents the conclusion of our study.

2 Related Work

After Google published the TCP-BBR, several studies have evaluated its performances under various network setups and conditions [15–17], while others have tried to improve various aspects of the BBR [8, 25–28].

According to Mahmud et al. [9], when more than two flows with various RTTs are involved in data transmission, BBR exhibits substantial throughput imbalance and packet loss rates. Their research led to the development of the BBR Advanced (BBR-A) algorithm, which reduces cwnd by half and transmission rate by 25% when congestion and packet loss are identified. Kim et al. [29] evaluation of the BBR algorithm reported that long RTT flows consistently suppress and outperform their short RTT counterparts. This eventually leads to throughput imbalance and queuing delay as long RTT traffic occupies a bigger portion of the available bandwidth. To solve the BBR mentioned above design problems, their study came up with an algorithm that restricts the inflight cap during probebw phase below 2 BDP when congestion is detected to minimize and suppress the existing queue; otherwise, it is reset and limited to 1 BDP. Longer RTT always flows probe for a longer frame time than short RTTs ones, which makes them dominate the generated queue backlogs, according to Ma et al.'s [16] analysis of the BBR algorithm. Therefore, by limiting and restricting the long duration of the long RTT traffic during the probing session, their work proposed a strategy that will lower the high dominance of long RTT flows. Fairness between the long and short RTT flows will result from this. Through their analysis, Kim et al. [20] found that the BBR typically sends too much data, leading to unjust bandwidth distribution among various flows and buffer overflow. As a result, they recommended the Delay-Aware BBR (DA-BBR) variant to boost throughput and RTT fairness design issues. Their algorithm regulated the data transmission rate based on the correlation between RTT and RTprop controlling parameters. According to Pan et al. [30], the majority of the proposed BBR variations algorithms still apply a fixed cwnd gain of 2 BDP to restrict the amount of data each flow could transmit. As a result, long RTT flows will always have a larger cwnd compared to short RTT ones. Therefore, they proposed the BBR-ACW variant, which modifies the cwnd gain adaptively based on the relationship between the delivery rate and the RTT. According to our earlier research [14], BBR computed BDP as BltBw * RTprob. Short RTT flows were constrained since

long RTT traffic has a larger RTprop value; hence long RTT flows now occupy a substantial portion of the available bandwidth. As a result, we introduced the method BBR-With enhanced RTT fairness (BBR-Efra), which adaptively modifies the BDP computation values.

To our knowledge, all published BBR variants blindly apply the fixed pacing rates of 1.25 and 0.75 when probing up for more available bandwidth and during probe down intervals, respectively, in the probebw phase. The above-described pacing rates generate a persistent queue based on our experimental results and other studies' evaluations analysis [8, 9]. Short RTT will be limited beforehand and progressively release bandwidth to the larger RTT flows since long RTT traffic has bigger RTprop values; therefore, their cwnd will be greater than short RTT flows'.

3 Motivation: BBR's RTT Fairness Issues

From our extensive experimental analysis, we revealed that during probebw phase, BBR blindly applies a pacing rate of 1.25 to probe up for more available bandwidth without considering factors like a buffer or even link queue statuses and sizes, followed by a 0.75 pacing rate to probe down to drain, clear queue and congestion generated during probe up. Finally, a pacing rate of 1 is applied for the remaining six RTT cycles to sustain the consistent and stable bandwidth. When a single TCP flow transverses through the bottleneck link, BBR converges at the right designed Kleinrock's optimal operating point, and in this case, it operates with the maximum expected delivery rate and with minimal delays. But, when several TCP flows with varying RTTs coexist on a common bottleneck link, the total data transmission rate usually exceeds the available bandwidth, resulting in excessive buffer overflows (bufferbloat). The default probe down pacing rate applied by BBR is not good enough to drain, clear, and consume the queue generated on the bottleneck buffer and on the link, resulting in a persistent queue backlog. Due to the excessive transmission rate and incapacity of releasing the queue, BBR operates at the point which has shifted slightly to the right of the Kleinrock operating point, as demonstrated in Figure 3. Based on the queuing theory, bottleneck queue sharing determines the network flow's throughput. Based on the queuing theory, the generated bottleneck queue sharing influences the network flow's throughput. When the queue is formed and, with time, becomes persistent, short RTT flows are prematurely constricted by cwnd, and the flows can not acquire additional bandwidth even with more frequent probes performed [8].

Moreover, the long RTT flows have bigger and higher estimated RTprop values, taking a larger fraction of the already formed persistent queue and occupying a more significant bandwidth consumption share. The high dominance of long RTT flows makes short RTT ones decrease the delivery rate of the subsequent bandwidth detection sessions; this makes the transmission rate of the short RTT flows smaller. In conclusion, when the bottleneck buffer and the link are overloaded, the longer RTT flows have a higher data-sending

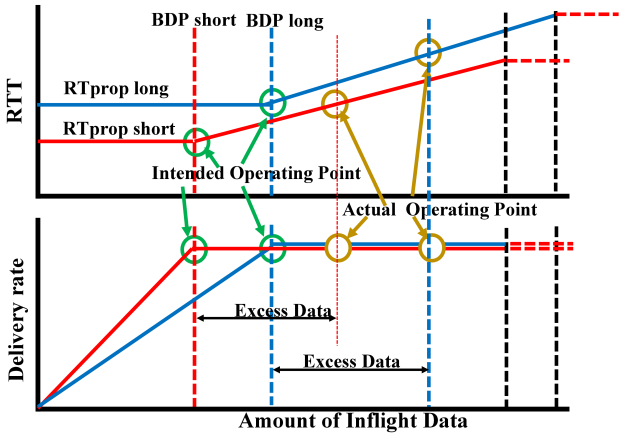


Fig. 3 Intended Vs. Actual operating points in TCP-BBR Algorithm

rate than their short RTT counterparts [8]. The high imbalanced throughput caused by the strong dominance of long RTT flows over short RTT flows is a serious network flaw that malevolent users can take advantage of by purposefully extending the delays in their flows to obtain an unfairly much bigger bandwidth share.

4 Proposed Algorithm: BBR-APG

When several TCP flows coexist on a common shared bottleneck link, BBR sends extra data and makes the total transmission rate exceed the bottleneck capacity, forming a queue that, with time, becomes a persistent queue. Based on our various experimental analyses, bottleneck buffer or link queue statuses and sizes must be considered before applying probe up and probe down pacing gain rates during the bandwidth estimation phase. The fixed pacing rate values offered by BBR and other BBR variants cause the data transferred by the BBR to surpass the link's capacity whenever the bottleneck buffer is smaller than 1 BDP. As a result, buffer overflows occur, which leads to persistent queue creation and problems with fairness amongst flows with various RTTs.

This study offered a new alternative solution to the TCP-BBR RTT fairness problem, which will substantially enhance it by adaptively regulating the probebw pacing rates and the upper limit of the cwnd based on different network congestion states using bottleneck inflight queue statuses and shifting the operating point of the BBR close to the intended operating point and to the position where the bottleneck buffer is not filled up. Our approach recorded a substantial throughput increment of short RTT flows that was severely suppressed.

Assume there are x number of BBR flows cruising through bottleneck link with an available bandwidth of B. Let $flow_i$ $(i \in [1, y])$ represent $flow_i$ and $r_i(t)$ express the delivery rate of the $flow_i$ at the time t. Let $T_i = q_i + p_i$ represent RTT of $flow_i$, while q_i represents queue delay of $flow_i$, and p_i is

RTprop (obtained in ProbeRTT phase). Let $L_i = r_i * (q_i + p_i)$ represent inflight data of $flow_i$ while $Q_i = r_i * p_i$; is the delivery capacity. The bottleneck link's queue balance is heavily influenced by q_i and r_i.

BBR-APG updates L_i and Q_i after every probebw RTT cycle to obtain the latest state of the bottleneck queue status based on different RTT flows.

Algorithm 1 describes the logic implementation of our proposed algorithm, BBR-APG. When the bottleneck link status becomes $L_i \leq 1.25Q_i$ and when $p_i > T_i$, the bottleneck buffer is either not occupied or very little occupied with the inflight data, and there is no queue detected on the link, respectively. Therefore, BBR can probe more available bandwidth with an innovative adaptive probe up pacing rate technique instead of the default fixed pacing rate value of 1.25 on different RTT flows to acquire more available bandwidth, which is essential during this interval. The adaptive probe up pacing rates provided by BBR-APG is influenced by inflight queue status; the smaller the queue size within the buffer, the bigger and larger the pacing rate value to be assigned and vice versa. In summary, the lesser the inflight queue, the larger the probe up pacing rate value to transmit more packets, and the bigger the inflight queue, the smaller the probe up pacing rate value to be assigned to transmit fewer packets to prevent bufferbloat phenomena. Our algorithm provided a well-thought-out and crafted equation, which not only perfectly fitted and provided suitable adaptive probe up pacing values but also values that ensure short RTT flows are not constricted by cwnd prematurely. But when the status turns to $L_i > 1.25Q_i$ and $T_i > p_i$, the network is overloaded; the bottleneck buffer is experiencing buffer overflows, and the link is congested. Therefore, BBR-APG can drain and clear the queue generated on the buffer and the link; our proposed algorithm applies an adaptive probe down pacing rates instead of the fixed default value of 0.75. BBR-APG determines the values for its adaptive probe down pacing rate based on the infight queue status. The bigger and longer the inflight queue, the lower the pacing rate values to be assigned and vice versa; this technique slows down the sending rate adaptively and clears the existing queue generated on the bottleneck buffer and on the link. The adaptive probe down pacing rate values provided by our algorithm is lower than the default value of 0.75, releasing the queue much quicker than the original BBR probe down pacing value. From our formulated equation, since long RTT flows have larger p_i, they will experience more P_{down} reduction than short RTT flows, thus guaranteeing improved fairness of the two competing flows when battling for the available bottleneck bandwidth. Unlike the original BBR algorithm, which blindly probes up and probes down using fixed rate values of 1.25 and 0.75, respectively, without considering factors like a buffer or link queue statuses, our proposed algorithm considers both buffer and link queue statuses before adaptively adjusting the pacing rates of both probe up and probe down sessions based on inflight queue status. Therefore, BBR-APG eliminates the creation of a persistent queue, increases the fairness between the competing flows and throughput, and reduces the packet loss rates.

Algorithm 1: BBR-APG

1. **INPUT:** $mode, r_i, T_i, p_i$
2. **OUTPUT:** $Pacing_Gain$
3. **if** $bbr \rightarrow mode = BBR_PROBE_BW$ then
4. **if** $(L_i \leq 1.25Q_i$ && $T_i \leq p_i)$ then
5. $Pacing_Gain(P_{up}) = \frac{(r_i+q_i)*L_i}{(p_i+q_i)*L_i}$
6. **else**
7. $Pacing_Gain == 1.0$
8. **End if**
9. **Else if**$(L_i > 1.25Q_i$ && $T_i > p_i)$
10. $Pacing_Gain(P_{down}) = \frac{(p_i+q_i)*L_i}{(r_i+q_i)*L_i}$
11. **else**
12. $Pacing_Gain == 1.0$
13. **End if**
14. **return** $Pacing_Gain$
15. **End if**

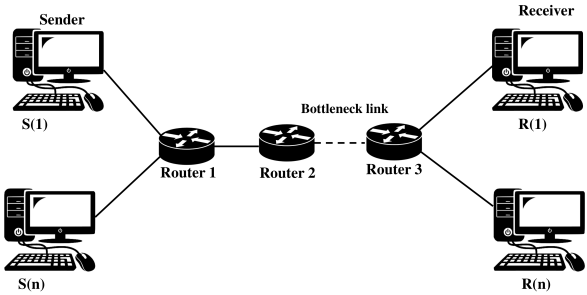


Fig. 4 NS3 simulation network topology.

5 Experimental setup and Results analysis

In this section, we discuss the implementation of our proposed methodology. We setup our experimental analysis using the NS3 simulator running on Linux Kernel version 4.14 to evaluate the performance of BBR-APG based on the topology demonstrated in Figure 4. According to our various experimental analyses, we simulated several parameters, including bottleneck bandwidth, RTTs, random packet loss rates, and bottleneck buffer queue sizes. Additionally, we used the DropTail active queue management (AQM) default setting and set the size of each packet to 1 kb. In order to facilitate pacing, we further altered the queuing discipline to fair queuing (fq), and each sender used iPerf3 to transfer the packets to the receiver. During this study, we searched and applied the most common and recently published BBR variants like BBR [31], DA-BBR [20], and BBR-ACW [30] as a baseline of evaluation performance analysis. All the BBR variants used in this study were deployed on the sender(server) side.

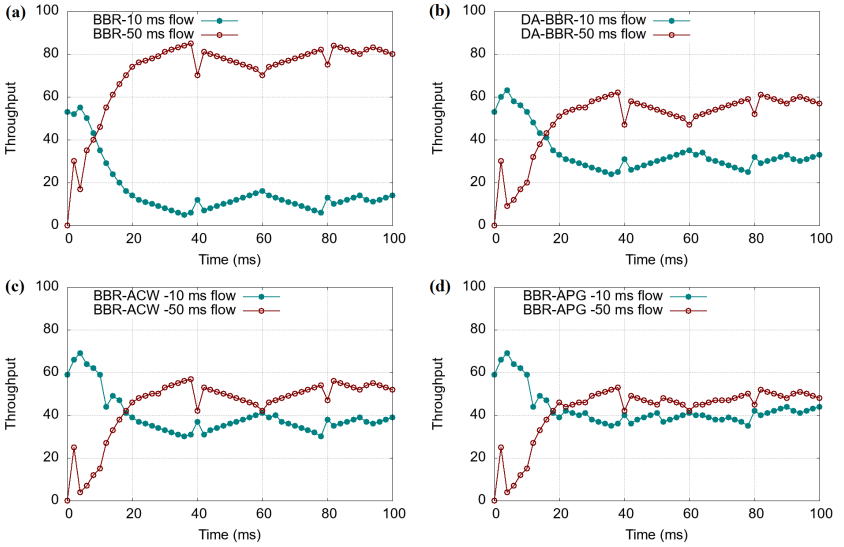


Fig. 5 Comparing average throughput of various BBR variants on different TCP flows (a)BBR (b)DA-BBR (c)BBR-ACW (d)BBR-APG

5.1 Evaluation Metrics

5.1.1 RTT Fairness

We conducted an RTT fairness evaluation of the performances of various BBR variants with respect to RTT changes. Where flows with varying RTTs competed for 100 Mbps bandwidth on a 5 BDP bottleneck buffer size. Our experiments evaluated BBR RTT fairness by comparing 10 ms Vs. 50 ms TCP RTT flows, as presented in Figure 5.

Figure 5(a) demonstrates that 10 ms flows hurriedly obtained high bandwidth at the startup phase. Once bandwidth normalizes, the average bandwidth of 10 ms flows was about 14 Mbps, while the 50 ms flows got an average bandwidth of 82 Mbps. This shows that BBR short RTT flows were significantly repressed by long RTT flows. The DA-BBR method demonstrated better fairness performance results than the traditional BBR algorithm, as depicted in Figure 5(b). Figure 5(c) BBR-ACW outperformed the standard BBR and DA-BBR, achieving a throughput differential between the two flows of 1.4 times. In Figure 5(d), BBR-APG outperforms all its competitors by achieving a throughput variance of 1.2 when the two different flows compete for the same bottleneck bandwidth. BBR-APG displayed much-desired aggressiveness and produced better fairness performance than other variants.

5.1.2 Bottleneck Bandwidth Utilization

We conducted another experimental analysis to determine each TCP flow's throughput to calculate and identify the available bottleneck link bandwidth utilization. Our experiment evaluated the effectiveness of our proposed

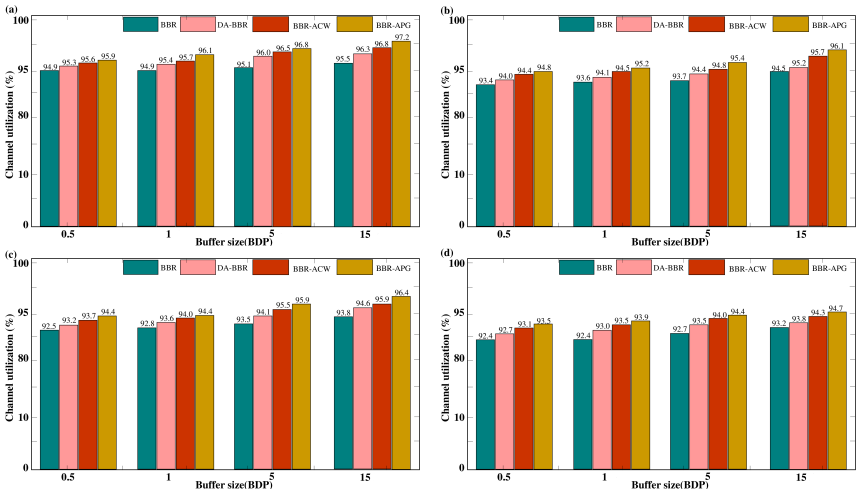


Fig. 6 Bottleneck bandwidth utilization of different BBR Variants with varying link loss rates: (a) Link packet loss rate: 0%. (b) Link packet loss rate: 1%. (c) Link packet loss rate: 3%. (d) Link packet loss rate: 5%.

approach against other recently published BBR variants on non-congested network scenarios ideal for achieving the highest link bandwidth utilization that can be attained under optimal network setups and conditions. We setup a single TCP flow scenario and simulate various random packet loss links (0%, 1%, 3%, and 5%), BDP buffer sizes (0.5 BDP, 1 BDP, 5 BDP, and 15 BDP), and 100 Mbps bottleneck bandwidth on NS3. The bottleneck bandwidth utilization for all the flows is computed as in Equation (1) [30].

$$n = \frac{\sum_i bytes_i}{Cap * duration} \tag{1}$$

Where $bytes_i$ is the total length of all received packets, Cap is the Btlbw, and duration is the time spent during stimulation (100 s)

Figure 6(a) represents a random packet loss rate of 0% where all algorithms were expected to perform at their best because this is the most optimal network condition we can have. All the algorithms evaluated recorded more than 94.9% of bottleneck bandwidth utilization. BBR algorithm acquired the least bandwidth utilization, while DA-BBR, BBR-ACW, and BBR-APG recorded improved bottleneck bandwidth utilization in shallow and deeper buffer setups. Our proposed algorithm outperformed other BBR variants by achieving the highest bandwidth utilization of 97.2%.

Figure 6(b)-(d) represent 1%, 3%, and 5% random link packet loss rates, respectively, whereby BBR recorded the lowermost bandwidth utilization in all evaluated scenarios, and BBR-ACW showed better bandwidth utilization than DA-BBR. BBR-APG registered the highest bandwidth utilization in all evaluated cases. All algorithms recorded a decrease in bandwidth utilization compared with the 0% random packet loss rate. However, our proposed algorithm was not significantly affected as it maintained a minimum bandwidth

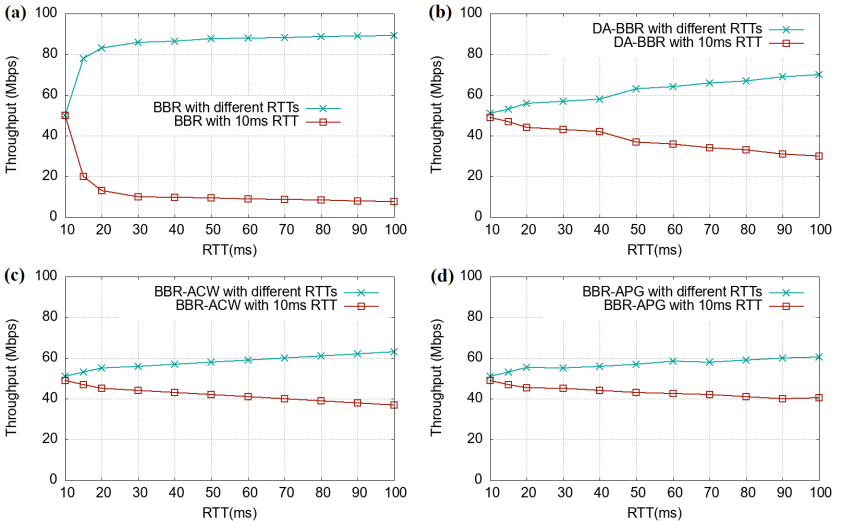


Fig. 7 Comparison of average throughput, when 10 ms RTT flows compete with other TCP flows on a 0.5 BDP buffer size: (a) BBR (b) DA-BBR (c) BBR-ACW (d) BBR-APG.

utilization of 93.5%. BBR-APG recorded and upheld the highest and most optimal bandwidth utilization, which is essential during download and upload sessions across various buffer sizes and random packet loss rates.

By making the best use of the available bandwidth, our suggested technique increases the overall throughput performance of the entire network.

5.1.3 Throughput

Figure 7(a)-(d) exhibits various performance changes in the evaluation analysis of the four different BBR variations when a short RTT flow (10 ms) competes against other flows with varying RTTs on a 100 Mbps shared bandwidth and a 0.5 BDP buffer size. We noticed that fairness tended to decline in several cases as the RTT ratio rose. Figure 7(a) shows a throughput between the two flows being more than 4.5 times when the RTT variation was about 30 ms, and short RTT flows only managed to get around 12% when the RTT variation surpassed five times (50 ms). The DA-BBR method is depicted in Figure 7(b); it recorded a notable fairness improvement when the RTT difference was below 50 ms. However, the DA-BBR algorithm saw long RTT flows overwhelming short RTT when the RTT differential exceeded five times (50 ms). Figure 7(c) BBR-ACW algorithm showed a good fairness performance even after the RTT difference exceeded six times (60 ms). Finally, Figure 7(d) displayed BBR-APG long RTT flows getting a throughput of about 52%, while short RTT flows obtained more than 45% bandwidth consumption even when 10 ms flows were in competition with 100 ms ones.

We carried out an additional experiment to confirm the efficacy of our suggested methodology, as shown in Figure 8.

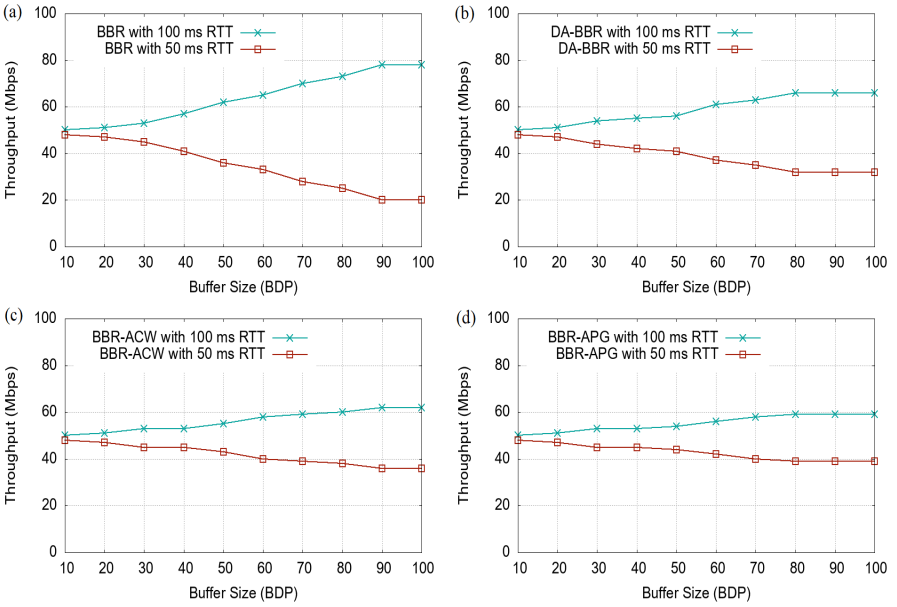


Fig. 8 Comparison of average throughput when 50 ms Vs. 100 ms RTT flows compete against each other under varying BDP bottle buffer sizes. (a) BBR (b) DA-BBR (c) BBR-ACW (d) BBR-APG.

Figure 8(a) demonstrates bandwidth distribution in BBR when 50 ms Vs. 100 ms flows were competing for 100 Mbps on varying bottleneck buffer sizes. The analysis shows as the buffer increases in size, the fairness widens. Compared to the BBR, the DA-BBR algorithm in Figure 8(b) performs significantly better regarding bandwidth distribution across the two flows. A steady throughput share above 38 Mbps for 50 ms RTT flows exists even when the buffer size exceeds 50 BDP. The BBR-ACW method, depicted in Figure 8(c), outperformed the BBR and DA-BBR algorithms regarding bandwidth sharing between the two flows. Even with a 100 BDP buffer capacity, it still revealed a throughput differential of around 1.5 times between the two flows. Figure 8(d) shows the BBR-APG performs better than its rivals by registering a throughput differential of around 1.2 times between the two flows even when a 100 BDP buffer queue size is used.

Our experimental evaluation analysis demonstrated BBR-APG's fairest performance when two competing flows coexist in a shared bottleneck link with varying bottleneck buffer queue sizes.

5.1.4 Jain's Fairness Index

The previous throughput evaluation analysis can not precisely reflect throughput changes and differences in some scenarios. The Jains fairness index measure was added to the equation to better assess BBR-APG's performance on RTT fairness. The evaluation metric indicates the fairness of bandwidth allocation

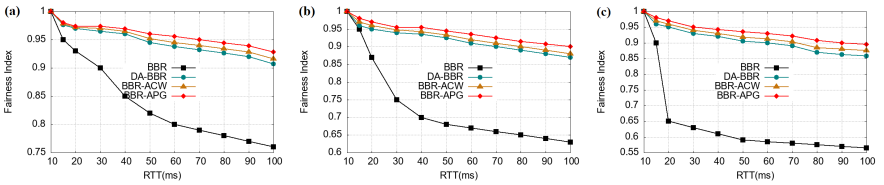


Fig. 9 Jain's fairness index comparative analysis when short flows (10 ms) coexist with a varying RTT flow: (a) Bandwidth: 10 Mbps. (b) Bandwidth: 50 Mbps. (c) Bandwidth: 100 Mbps.

when two different RTT flows are competing [18, 32]. Jain's fairness index is computed as in Equation (2) [30].

$$Z = \left(\frac{\left(\sum_{i=1}^{n} x_i \right)^2}{n \sum_{i=1}^{n} {x_i}^2} \right) \tag{2}$$

Fairness in bandwidth distribution is indicated when Z is close to 1.

Figure 9 shows Jain's fairness evaluation analysis of the four TCP-BBR variants when competing for various bottleneck bandwidths on 1 BDP buffer size.

When various flows were competing for 10 Mbps, Figure 9(a) reveals that BBR reported the lowest fairness index of roughly 0.72. DA-BBR and BBR-ACW have a fairness index of 0.90 and 0.91, respectively, a notable improvement from the original version TCP-BBR. BBR-APG outperformed other algorithms by registering a fairness index of 0.93 even when 10 ms is competing against 100 ms RTT flows. Figure 9(b) recorded a fairness decline for all the algorithms evaluated compared to Figure 9(a). BBR recorded the least fairness of about 0.63. When the RTT variation was about four times, Jain's fairness of DA-BBr and BBR-ACW were about 0.93 and 0.94, respectively, while BBR-APG recorded 0.96. But when competing against 100 ms flows, DA-BBR showed a significant drop and recorded a minimum of 0.86, while BBR-ACW recorded a fairness index of 0.88. BBR-APG showed a better and improved fairness index of 0.91. In Figure 9(c), when different RTT flows share 100 Mbps bottleneck bandwidth, BBR obtained a fairness index of only 0.56. DA-BBR showed better performance compared to BBR but not good as in 10 Mpbs bottleneck bandwidth and received a fairness index of 0.84, while BBR-ACW showed fairness of about 0.86. BBR-APG outperformed other algorithms with a fairness of 0.90.

BBR-APG managed to obtain a fairness index above 0.9 between two TCP flows with different RTTs, a massive improvement from the standard version of the BBR algorithm.

5.1.5 Retransmissions

Because BBR adaptively adjusts its probebw phase pacing rates based on the network congestion state and the ability of our proposed methodology P_{down} pacing rate to drain and clear the buffer and link congestion, which eliminates

the persistent queue formation, unlike the original BBR algorithm, BBR-APG was able to reduce packet loss rates and greatly reduces packet retransmission rates. The effectiveness of the congestion control algorithm increases with decreasing registered packet loss and retransmission rates.

Figures 10(a) and 10(b) show the results of an experiment we ran utilizing bottleneck bandwidths of 10 Mbps and 100 Mbps, respectively, to assess the performance of our proposed method on 5 BDP buffer size where short RTT flow (10 ms) coexist with other varying TCP flows when sharing the available bottleneck bandwidth. Figure 10(a) demonstrates that the number of retransmissions increases with decreasing RTT difference, and most of these retransmissions occur at the startup phase. As the RTT difference between the competing flows increases, the retransmission rates decrease for all the algorithms evaluated. BBR-APG recorded the lowest retransmission rate for all the existing flows. A similar observation was recorded in Figure 10(b), where BBR displays a hefty retransmission rate while other algorithms showed a relatively reduced rate.

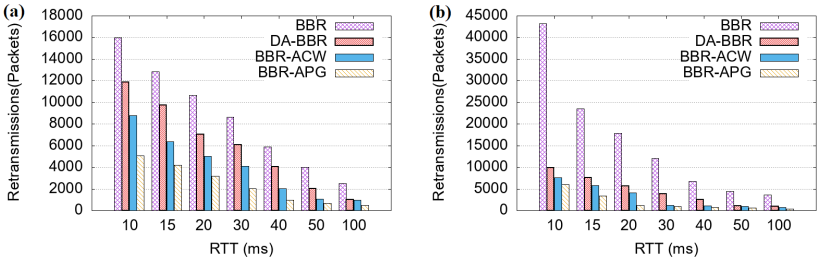


Fig. 10 Retransmissions rate when 10 ms RTT flow coexists with a different RTT flow: (a) Bandwidth: 10 Mbps. (b) Bandwidth: 100 Mbps.

BBR-APG maintained the bottommost retransmission rate, outperforming other state-of-the-art algorithms in all evaluated cases, regardless of bandwidth, due to its adaptive probe down pacing rate technique, which drains the existing buildup queue and prevents excessive packet loss and retransmissions.

5.2 Latency

We conducted another experimental analysis involving several TCP flows with varying RTTs coexisting on the same bottleneck link. During this experimental analysis, we simulated 100 Mbps as the bottleneck bandwidth and a buffer queue size of 10 BDP. Figure 11 shows that when only single TCP traffic is involved, all techniques registered low queuing and latency delays because all the BBR variants converged and operated at the designated operating point. As the TCP competing flows surged, the BBR algorithm recorded high queuing latency delays due to buffer overflows (bufferbloat) occurrences experienced by the BBR, resulting in packet drops, hence the delays. Compared to the standard BBR, DA-BBR, and BBR-ACW exhibited significantly improved

latency and queuing delays. Despite the quantity of TCP flows applied, BBR-APG maintained minimal latency delay. BBR-APG demonstrated yet another crucial aspect of minimizing latency and queuing delays which is essential with delay-sensitive applications and services like real-time gaming, streaming, and VOIP.

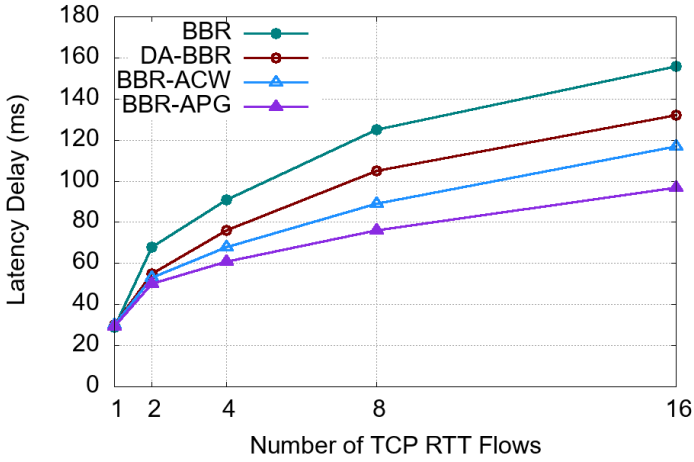


Fig. 11 Comparing RTT queuing delays for various TCP flows

Based on our experimental analysis and findings, we can certainly conclude that BBR-APG exhibits much-desired aggressiveness that significantly enhances the fairness of competition between various TCP flows with various RTTs. Comparing BBR, BBR-ACW, and BBR-APG, the latter showed more than three times and 18%, respectively, of RTT fairness improvement on different flows. Despite having much superior RTT fairness outcomes, BBR-APG also experienced a throughput increase of nearly 22% over BBR-ACW and a reduction in packet loss rates of more than 52% and 24% over BBR and BBR-ACW, respectively. Additionally, BBR-APG recorded significantly increased bottleneck link bandwidth utilization and decreased latency delay time.

6 Conclusion

The primary reasons for the RTT unfairness in the BBR congestion control algorithm were examined in this work. When two flows with differing RTTs share and coexist on a single bottleneck link, we noticed a severe unfairness of allocating bandwidth, where BBR advantages and favors long RTT flows over short RTT flows. This preference nature of BBR is the primary source of imbalanced throughput and a serious BBR flaw that some users can easily take advantage of by inducing delays in their flows to create unwarranted long RTT flows to unfairly enjoy the lion's share of the available bandwidth than the

rest of the users. We suggested the BBR-APG algorithm to address this fundamental BBR design problem. Our proposed approach adaptively adjusts the probebw phase pacing gain rates of each flow based on the inflight queue status, unlike the original BBR, which blindly probes up and probes down using fixed values of 1.25 and 0.75, respectively. With BBR-APG, multiple TCP flow with different RTT delays can fairly compete for valuable network resources such as bandwidth. Our proposed technique outperformed other popular BBR variants in all metrics evaluated in shallow and deeper buffer settings and on various bottleneck bandwidth setups. In comparison with the BBR algorithm, our proposed methodology achieved RTT fairness improvements of over 58%, throughput improvements of over 45% and packet retransmission reductions of over 52%.

Author Contributions Author Contributions Charles Kihungi Njogu: Investigation, Software, Visualization, and Writing original draft. Wang Yang: Supervision, Conceptualization, Validation, and Formal Analysis review. Humphrey Waita Njogu: Validation, Writing Review & Editing, and, finally, Methodology. Adrian Bosire: Conceptualization, Writing the original draft, and Software.

Funding This research was supported in part by the National Natural Science Foundation of China (Grants No.62072474), in part by the Natural Science Foundation of Hunan Province (Grants No.2021JJ30874).

Data Availability Not applicable.

Declarations

Ethics Approval Not applicable.

Consent to Publish All authors have approved the published version of the manuscript.

Conflicts of Interest All authors declare that they have no potential competing for financial interests or personal relationships that could have appeared to influence the work reported in this study.

References

[1] Floyd, S., Handley, M., Padhye, J.: A comparison of equation-based and aimd congestion control (2000)

[2] Yang, Y.R., Lam, S.S.: General aimd congestion control. In: Proceedings 2000 International Conference on Network Protocols, pp. 187–198 (2000). IEEE

[3] Floyd, S., Henderson, T., Gurtov, A., et al.: The newreno modification to tcp's fast recovery algorithm (1999)

[4] Xu, L., Harfoush, K., Rhee, I.: Binary increase congestion control (bic) for fast long-distance networks. In: IEEE INFOCOM 2004, vol. 4, pp. 2514–2524 (2004). IEEE

[5] Ha, S., Rhee, I., Xu, L.: Cubic: a new tcp-friendly high-speed tcp variant. ACM SIGOPS operating systems review **42**(5), 64–74 (2008)

[6] Yang, P., Zhang, N., Bi, Y., Yu, L., Shen, X.S.: Catalyzing cloud-fog interoperation in 5g wireless networks: An sdn approach. IEEE Network **31**(5), 14–20 (2017)

[7] Yang, P., Zhang, N., Zhang, S., Yu, L., Zhang, J., Shen, X.: Content popularity prediction towards location-aware mobile edge caching. IEEE Transactions on Multimedia **21**(4), 915–929 (2018)

[8] Pan, W., Li, X., Tan, H., Xu, J., Li, X.: Improvement of rtt fairness problem in bbr congestion control algorithm by gamma correction. Sensors **21**(12), 4128 (2021)

[9] Mahmud, I., Kim, G.-H., Lubna, T., Cho, Y.-Z.: Bbr-acd: Bbr with advanced congestion detection. Electronics **9**(1), 136 (2020)

[10] Najmuddin, S., Asim, M., Munir, K., Baker, T., Guo, Z., Ranjan, R.: A bbr-based congestion control for delay-sensitive real-time applications. Computing **102**, 2541–2563 (2020)

[11] Cardwell, N., Cheng, Y., Gunn, C.S., Yeganeh, S.H., Jacobson, V.: Bbr: congestion-based congestion control. Communications of the ACM **60**(2), 58–66 (2017)

[12] Grazia, C.A., Patriciello, N., Klapez, M., Casoni, M.: Bbr+: improving tcp bbr performance over wlan. In: ICC 2020-2020 IEEE International Conference on Communications (ICC), pp. 1–6 (2020). IEEE

[13] Miyazawa, K., Sasaki, K., Oda, N., Yamaguchi, S.: Cycle and divergence of performance on tcp bbr. In: 2018 IEEE 7th International Conference on Cloud Networking (CloudNet), pp. 1–6 (2018). IEEE

[14] Njogu, C.K., Yang, W., Njogu, H.W., Bosire, A.: Bbr-with enhanced fairness (bbr-efra): A new enhanced rtt fairness for bbr congestion control

algorithm. Computer Communications (2022)

[15] Hock, M., Bless, R., Zitterbart, M.: Experimental evaluation of bbr congestion control. In: 2017 IEEE 25th International Conference on Network Protocols (ICNP), pp. 1–10 (2017). IEEE

[16] Ma, S., Jiang, J., Wang, W., Li, B.: Fairness of congestion-based congestion control: Experimental evaluation and analysis. arXiv preprint arXiv:1706.09115 (2017)

[17] Zhang, S.: An evaluation of bbr and its variants. arXiv preprint arXiv:1909.03673 (2019)

[18] Jain, V., Mittal, V., Tahiliani, M.P.: Design and implementation of tcp bbr in ns-3. In: Proceedings of the 10th Workshop on Ns-3, pp. 16–22 (2018)

[19] Prakash, M., Abdrabou, A.: On the fidelity of ns-3 simulations of wireless multipath tcp connections. Sensors **20**(24), 7289 (2020)

[20] Kim, G.-H., Cho, Y.-Z.: Delay-aware bbr congestion control algorithm for rtt fairness improvement. IEEE Access **8**, 4099–4109 (2019)

[21] Cardwell, N., Cheng, Y., Yeganeh, S.H., Swett, I., Vasiliev, V., Jha, P., Seung, Y., Mathis, M., Jacobson, V.: Bbrv2: A model-based congestion control. In: Presentation in ICCRG at IETF 104th Meeting (2019)

[22] Sasaki, K., Hanai, M., Miyazawa, K., Kobayashi, A., Oda, N., Yamaguchi, S.: Tcp fairness among modern tcp congestion control algorithms including tcp bbr. In: 2018 IEEE 7th International Conference on Cloud Networking (CloudNet), pp. 1–4 (2018). IEEE

[23] Kfoury, E.F., Gomez, J., Crichigno, J., Bou-Harb, E.: An emulation-based evaluation of tcp bbrv2 alpha for wired broadband. Computer Communications **161**, 212–224 (2020)

[24] Aulbach, J.: Evaluation of tcp bbr v2 congestion control using network emulation

[25] Zhang, Y., Cui, L., Tso, F.P.: Modest bbr: Enabling better fairness for bbr congestion control. In: 2018 IEEE Symposium on Computers and Communications (ISCC), pp. 00646–00651 (2018). IEEE

[26] Kim, G.-H., Song, Y.-J., Cho, Y.-Z.: Improvement of inter-protocol fairness for bbr congestion control using machine learning. In: 2020 International Conference on Artificial Intelligence in Information and Communication (ICAIIC), pp. 501–504 (2020). IEEE

[27] Su, B., Jiang, X., Jin, G., Chen, H.: Rethinking the rate estimation of bbr congestion control. Electronics Letters **56**(5), 239–241 (2020)

[28] Song, Y.-J., Kim, G.-H., Cho, Y.-Z.: Bbr-cws: improving the inter-protocol fairness of bbr. Electronics **9**(5), 862 (2020)

[29] Kim, G.-H., Mahmud, I., Cho, Y.-Z.: Fairness improvement of bbr congestion control algorithm for different rtt flows. In: 2019 International Conference on Electronics, Information, and Communication (ICEIC), pp. 1–2 (2019). IEEE

[30] Pan, W., Tan, H., Li, X., Li, X.: Improved rtt fairness of bbr congestion control algorithm based on adaptive congestion window. Electronics **10**(5), 615 (2021)

[31] Cardwell, N., Cheng, Y., Gunn, C.S., Yeganeh, S.H., Jacobson, V.: Bbr: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time. Queue **14**(5), 20–53 (2016)

[32] Jain, R.K., Chiu, D.-M.W., Hawe, W.R., et al.: A quantitative measure of fairness and discrimination. Eastern Research Laboratory, Digital Equipment Corporation, Hudson, MA (1984)